

09 — Lời thoại quay video

Đọc theo file này. Phần **in nghiêng** là thao tác, phần trong ngoặc kép là câu nói. Không cần thuộc lòng, đọc theo cũng được — miễn hiểu mình đang nói gì.

Tổng khoảng **12 phút**.

Mở đầu — 40 giây

Chiếu màn hình landing page.

"Em chào thầy. Em làm website bán một đầu sách, tên là Nhà Giả Kim.

Kiến trúc em tách làm hai phần: backend là ASP.NET Core Web API viết bằng C#, frontend là React gọi sang backend qua HTTP. Database em dùng SQL Server.

Hôm nay em không trình bày giao diện, em sẽ đi theo **luồng dữ liệu**. Em sẽ bấm một nút, rồi chỉ cho thầy xem dữ liệu đi qua những chỗ nào, code nào chạy, và cuối cùng nó nằm ở đâu trong database."

PHẦN 1 — Khách đặt hàng (6 phút)

1.1 — Xem database trước khi làm gì — 1 phút

Chuyển sang SSMS. Chạy câu đầu tiên.

```
SELECT name AS Bang FROM sys.tables WHERE name <> '__EFMigrationsHistory' ORDER BY name;
```

"Trước hết em cho thầy xem database. Em có 10 bảng.

Trên đề thầy vẽ 5 nhóm chức năng, em tách thành 10 bảng theo chuẩn 1NF. Ví dụ nhóm Thông tin Sách, trong đó có mục Ảnh sách và Báo chí — một cuốn sách có **nhều** ảnh và **nhều** trích dẫn báo chí, nên em phải tách ra thành bảng riêng. Nếu để chung thì em phải nhét cả danh sách vào một cột, như vậy là vi phạm chuẩn 1NF ạ."

Chạy câu tiếp.

```
SELECT COLUMN_NAME, DATA_TYPE, NUMERIC_PRECISION, NUMERIC_SCALE  
FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'Orders' ORDER BY ORDINAL_POSITION;
```

"Đây là cấu trúc bảng Orders, nơi lưu đơn hàng.

Thầy chú ý giúp em hai cột UnitPrice và TotalPrice, em để kiểu **decimal 18 chấm 2**, không dùng float. Vì float là số thực dấu phẩy động, cộng dồn nhiều lần sẽ bị sai số. Tiền thì không được sai một đồng nào ạ."

Chạy câu đếm.

```
SELECT COUNT(*) AS SoDonHienTai FROM Orders;
```

"Hiện tại đang có N đơn. Em ghi nhớ con số này, lát nữa đặt xong mình quay lại xem."

1.2 — Bấm nút, xem nó gửi gì — 1 phút 30

Chuyển sang Chrome. Nhấn F12 mở DevTools, chọn tab Network. Cuộn xuống form đặt hàng.

"Em mở tab Network của trình duyệt để xem khi bấm nút thì nó gửi cái gì lên server."

Điền form: Họ tên, SĐT, Địa chỉ, Số lượng 3.

"Em điền thông tin: họ tên, số điện thoại, địa chỉ, số lượng 3 cuốn. Phương thức thanh toán em để COD."

Bấm "Xác nhận đặt hàng". Chờ màn hình cảm ơn hiện ra.

"Em bấm Xác nhận đặt hàng. Trang hiện ra màn hình cảm ơn kèm mã đơn."

Click vào dòng `orders` trong tab Network, chọn tab Payload hoặc Request.

"Đây là request mà trình duyệt vừa gửi đi. Phương thức là **POST**, địa chỉ là **gạch api gạch orders**. Content-Type là application/json.

Và đây là nội dung gửi lên: họ tên, số điện thoại, địa chỉ, số lượng 3, phương thức thanh toán.

Thầy chú ý giúp em một chỗ: trong này **hoàn toàn không có giá tiền**. Client chỉ được nói là mua mấy cuốn thôi, không được nói giá bao nhiêu. Lát nữa em giải thích vì sao."

Click tab Response.

"Và đây là cái server trả về: mã đơn, đơn giá 69 nghìn, tổng tiền 207 nghìn. Status là **201 Created**."

1.3 — Vào code frontend xem nút làm gì — 1 phút

Chuyển sang VS Code. Mở `frontend/src/components/landing/OrderFormSection.tsx`.

"Giờ em vào code xem cái nút đó làm gì."

Kéo tới dòng 221.

"Đây là cái nút, nó là `type submit` nằm trong thẻ form."

Kéo lên dòng 105.

"Khi bấm, sự kiện submit được `handleSubmit` của thư viện react-hook-form bắt lại. Nó chặn hành vi mặc định của trình duyệt — tức là không load lại trang — rồi gom giá trị các ô thành một object và gọi `mutation.mutate`."

Kéo lên dòng 13.

"Trước khi gửi, dữ liệu phải qua lớp kiểm tra này. Em dùng thư viện Zod. Ví dụ số điện thoại phải khớp biểu thức chính quy này — bắt đầu bằng số 0 và đủ 10 chữ số. Số lượng phải từ 1 đến 99. Sai thì nó hiện lỗi đỏ ngay dưới ô và **không gửi request nào** lên server. Nhưng em xin nói rõ: kiểm tra ở đây chỉ để người dùng biết mình gõ sai thôi, **không phải để bảo mật**. Ai mở DevTools lên cũng bỏ qua được hết. Chốt chặn thật nằm ở server ạ."

Mở `frontend/src/api/endpoints.ts`, dòng 13.

"Còn đây là chỗ thật sự gửi đi. Em dùng thư viện axios, gọi `api.post` tới đường dẫn gạch `api/gach/orders`."

1.4 — Sang backend: request vào controller nào — 1 phút

Mở `backend/NhaGiaKim.Api/Controllers/OrdersController.cs`.

"Bây giờ sang phía server. Request vừa nãy đi vào controller này."

Chỉ dòng 9.

"Dòng 9: Route `api/gach/orders` — khớp đúng cái đường dẫn mình vừa thấy trong tab Network."

Chỉ dòng 13.

"Dòng 13: `HttpPost` — tức là nó nhận phương thức POST."

Chỉ dòng 14.

"Dòng 14 là rate limit, em giới hạn 5 đơn một phút trên mỗi địa chỉ IP để chống spam."

Chỉ dòng 18.

"Và đây là hàm xử lý. Thầy thấy nó rất ngắn: nhận request rồi gọi `orderService.CreateAsync`, xong đổi kết quả thành mã HTTP.

Controller **không chứa một dòng nghiệp vụ nào**. Em tách như vậy để logic nghiệp vụ có thể test được mà không cần dựng cả web server lên."

1.5 — Service xử lý — 2 phút ★ phần quan trọng nhất

Mở `backend/NhaGiaKim.Application/Services/OrderService.cs`.

"Đây là service, nơi chứa logic nghiệp vụ thật."

Chỉ dòng 15.

"Dòng 15, thầy để ý constructor này nhận vào `IAppDbContext`, `TimeProvider`. Service không tự tạo ra chúng, mà khai báo là mình cần rồi hệ thống tiêm vào. Đây là Dependency Injection, em đăng ký ở file `DependencyInjection.cs`."

Chỉ dòng 23 đến 27.

"Đầu tiên nó truy vấn xuống database lấy cuốn sách đang mở bán."

Chỉ dòng 34 và 35. Dừng lại ở đây, nói chậm.

"Đây là hai dòng quan trọng nhất của toàn bộ bài.

Dòng 34: đơn giá được lấy bằng cách đọc giá **từ database** — giá gốc 89 nghìn, giá giảm 69 nghìn, hàm này chọn ra giá bán thực tế là 69 nghìn.

Dòng 35: tổng tiền bằng đơn giá nhân số lượng. 69 nghìn nhân 3 bằng 207 nghìn.

Nghĩa là **server tự tính giá**, nó không hề dùng giá client gửi lên. Lúc này thầy thấy trong request không có giá — kể cả khách có cố tình gửi giá lên thì server cũng bỏ qua.

Nếu không làm vậy thì khách chỉ cần mở DevTools, sửa con số trong request, là mua được cuốn sách giá 1 đồng ạ."

Chỉ dòng 42.

"Dòng 42 sinh mã đơn, dạng NGK gạch ngày tháng gạch 4 ký tự ngẫu nhiên."

Chỉ dòng 57 và 60.

"Dòng 57: `db.Orders.Add` — dòng này mới chỉ đánh dấu là 'sẽ thêm', chưa chạm vào database.

Dòng 60: `SaveChangesAsync` — tới đây Entity Framework mới sinh ra câu SQL và gửi xuống database."

1.6 — Xem câu SQL thật — 45 giây

Chuyển sang cửa sổ terminal đang chạy `dotnet run` — câu SQL in thẳng ở đó.

"Em bật log để thấy xem câu SQL mà Entity Framework sinh ra. Đây ạ:"

```
INSERT INTO [Orders] ([Address], [BookId], [CreatedAt], [CustomerName], [Note],  
                      [OrderCode], [PaymentMethod], [Phone], [Quantity], [Status],  
                      [TotalPrice], [UnitPrice], [UpdatedAt])  
OUTPUT INSERTED.[Id]  
VALUES (@p0, @p1, @p2, ...);
```

"Em không viết SQL bằng tay. Em chỉ viết `Add` với `SaveChangesAsync` bằng C#, Entity Framework dịch ra câu INSERT này.

Thầy để ý nó dùng tham số `@p0`, `@p1` chứ không nối chuỗi. Nhờ vậy không bị SQL injection ạ."

1.7 — Record trong database — 45 giây

Chuyển sang SSMS.

```
SELECT TOP 3 Id, OrderCode, CustomerName, Phone, Quantity,  
             UnitPrice, TotalPrice, PaymentMethod, Status, CreatedAt  
FROM Orders ORDER BY Id DESC;
```

"Và đây là đơn vừa đặt, đã nằm trong database.

Đơn giá 69 nghìn, số lượng 3, tổng tiền 207 nghìn — đúng bằng cái server vừa tính ở dòng 35 lúc nãy.

Cột `PaymentMethod` bằng 0 là COD, cột `Status` bằng 0 là trạng thái Mới. Em lưu enum dưới dạng số cho gọn, phần ánh xạ em khai trong file `OrderConfiguration.cs`."

PHẦN 2 — Admin đăng nhập (3 phút)

2.1 — Bấm đăng nhập — 45 giây

Vào `/admin/login`, mở sẵn tab *Network*, điền email và mật khẩu, bấm *Đăng nhập*.

Click request `login` trong *Network*.

"Em đăng nhập. Request gửi lên là POST tới gạch api gạch auth gạch login, nội dung gồm email và mật khẩu.

Mật khẩu đi trong phần body chứ không nằm trên đường dẫn — nếu để trên URL thì nó bị ghi vào lịch sử trình duyệt và log của server ạ."

Click tab *Response*.

"Server trả về một chuỗi dài gọi là access token, kèm thời điểm hết hạn và thông tin người dùng."

2.2 — Code xử lý đăng nhập — 1 phút 15

Mở `backend/NhaGiaKim.Application/Services/AuthService.cs`.

Chỉ dòng 33 đến 35.

"Service truy vấn xuống bảng `AppUsers` tìm theo email. Chỗ `Include` này được dịch thành câu `INNER JOIN` sang bảng `Roles`, để lấy luôn vai trò trong một lần truy vấn."

Chỉ dòng 39 và 40.

"Rồi nó kiểm tra mật khẩu bằng hàm `Verify` của thư viện `BCrypt`."

Chuyển sang *SSMS*, chạy:

```
SELECT u.Id, u.Email, u.FullName, r.Name AS Role,
       LEFT(u.PasswordHash, 30) + '...' AS PasswordHash
FROM AppUsers u JOIN Roles r ON r.Id = u.RoleId;
```

"Thầy xem trong database, em **không lưu mật khẩu gốc**, chỉ có cột `PasswordHash` thôi. Chuỗi này bắt đầu bằng `2a` — đó là định dạng của BCrypt, số 12 là work factor, tức là độ khó khi băm. BCrypt là hàm một chiều, có salt riêng cho từng mật khẩu. Nên hai người đặt mật khẩu giống hệt nhau vẫn ra hai chuỗi hash khác nhau, và lộ database cũng không dò ngược ra mật khẩu được ạ."

Quay lại VS Code, chỉ dòng 39 lần nữa.

"Còn dòng này có một chi tiết nhỏ nhưng quan trọng: kể cả khi email **không tồn tại**, em vẫn chạy một phép BCrypt với một hash giả."

Vì nếu không làm vậy, email sai sẽ trả lời trong 2 mili giây, còn email đúng mà sai mật khẩu thì mất 300 mili giây. Người tấn công chỉ cần bấm giờ là biết email nào có thật trong hệ thống."

2.3 — Token và vai trò — 1 phút

Mở `backend/NhaGiaKim.Infrastructure.Security.JwtTokenGenerator.cs`, dòng 20.

"Đây là chỗ sinh token. Token chứa các thông tin gọi là claim: `sub` là id người dùng, `email`, và quan trọng nhất là `role` — vai trò Admin hay Staff."

Token được ký bằng thuật toán HMAC-SHA256, với khoá bí mật nằm trong biến môi trường chứ không nằm trong code."

Mở trình duyệt, vào jwt.io, dán token vừa copy từ tab Network.

"Em dán token vào đây để giải mã cho thầy xem."

Thầy thấy phần payload: `role` bằng `Admin`. Lấy `exp` trừ `nbf` ra đúng 3600 giây, tức token sống 60 phút.

Có một điều quan trọng: JWT chỉ **mã hoá base64** chứ **không phải mã hoá bí mật** — em vừa đọc được nội dung của nó ngay trước mặt thầy. Cái bảo vệ nó là **chữ ký**, chứ không phải sự che giấu. Sửa một ký tự trong đó là chữ ký không khớp, server từ chối ngay.

Chính vì ai cũng đọc được nên em không bao giờ bỏ mật khẩu vào trong token ạ."

Mở `frontend/src/api/client.ts`, dòng 12.

"Phía frontend, token được lưu vào `localStorage` . Rồi đoạn interceptor này của axios sẽ **tự gắn** header `Authorization Bearer` vào mọi request sau đó. Em không phải viết tay ở từng chỗ gọi API."

PHẦN 3 — Danh sách đơn trong admin (2 phút)

Vào `/admin/orders`, mở tab *Network*, click request `orders`.

"Khi vào trang đơn hàng, nó gọi GET tới gạch api gạch admin gạch orders, kèm tham số page và pageSize.

Thầy để ý trong Request Headers có `Authorization Bearer` — đây chính là token lúc này, do interceptor tự gắn vào."

Mở `backend/NhaGiaKim.Api/Controllers/Admin/AdminOrdersController.cs`, dòng 12.

"Controller này có attribute `Authorize` với policy `RequireStaffOrAdmin`. Nghĩa là phải có token hợp lệ, và vai trò phải là Admin hoặc Staff thì mới vào được."

Mở `backend/NhaGiaKim.Application/Services/AdminOrderService.cs`, dòng 32 đến 47.

"Đây là phần lọc. Mỗi điều kiện là một mệnh đề `Where` cộng dồn vào cùng một `IQueryable`.

Điều quan trọng là **tới đây chưa có câu truy vấn nào chạy cả**. Entity Framework mới chỉ dựng cây biểu thức trong bộ nhớ thôi."

Chỉ dòng 50 và 52 đến 54.

"Khi gặp `CountAsync` và `ToListAsync` ở đây thì nó mới dịch toàn bộ thành một câu SQL rồi gửi xuống database.

`Skip` và `Take` này được dịch thành `OFFSET` và `FETCH NEXT` trong SQL. Nhờ vậy việc lọc và phân trang làm ở **database**, chứ không phải tải hết về rồi lọc trong bộ nhớ. Nếu có mười nghìn đơn mà tải hết về thì trình duyệt treo ạ."

Chuyển sang terminal log, chiếu câu `SELECT`.

"Đây là câu SQL thật nó sinh ra."

CHỨNG MINH — 1 phút ★ nên làm

Mở terminal, dán lệnh này.

```
curl -X POST http://localhost:5173/api/orders -H "Content-Type: application/json" -d '{"custom
```

"Bây giờ em chứng minh cái em nói lúc nãy. Em gọi thẳng API, cố tình gửi kèm tổng tiền **bằng 1 đồng**, bỏ qua hết giao diện.

Và kết quả trả về vẫn là **207 nghìn**. Server bỏ qua hoàn toàn con số em gửi lên, nó tự đọc giá từ database rồi tính lại.

Đây là lý do vì sao mọi kiểm tra quan trọng đều phải đặt ở server ạ."

Dán lệnh thứ hai.

```
curl -i -X POST http://localhost:5173/api/orders -H "Content-Type: application/json" -d '{"cus
```

"Còn đây em gửi số điện thoại sai định dạng, thiếu số 0 đầu. Server trả về **400 Bad Request** kèm thông báo lỗi ở trường phone. Và trong database **không có bản ghi nào được tạo** ạ."

KẾT — 45 giây

Mở `docs/03-plan.md`, chiếu sơ đồ kiến trúc.

"Tóm lại kiến trúc của em: React gọi sang Web API, trong backend thì Controller gọi Service, Service gọi Entity Framework, Entity Framework nói chuyện với SQL Server.

Backend em chia thành 4 project: `Domain` chứa các lớp thực thể thuần; `Application` chứa logic nghiệp vụ; `Infrastructure` lo phần kỹ thuật như Entity Framework, BCrypt, JWT; `Api` chỉ có controller và cấu hình.

Nguyên tắc xuyên suốt bài của em là: **server không tin dữ liệu client gửi lên**. Giá do server tính, định dạng do server kiểm, quyền truy cập do server chặn. Giao diện chỉ là lớp hiển thị thôi ạ.

Em cảm ơn thầy."

Nhắc trước khi bấm ghi

Việc	Lệnh / thao tác
Bật backend	<code>ASPNETCORE_ENVIRONMENT=Development dotnet run --project backend/NhaGiaKim.Api --urls http://localhost:5080 --no-launch-profile</code>
Bật frontend	<code>npm --prefix frontend run dev</code> → mở http://localhost:5173
Xem log SQL	Nhìn thẳng vào cửa sổ terminal đang chạy <code>dotnet run</code> — câu SQL hiện ra ngay
Xem database	SSMS → <code>(localdb)\MSSQLLocalDB</code> → Windows Authentication (không cần mật khẩu)
Phóng to chữ VS Code	<code>Ctrl + +</code> vài lần, cỡ 16–18
Phóng to chữ SSMS	Tools → Options → Fonts and Colors, cỡ 14–16
Mở sẵn 5 file	<code>OrderFormSection.tsx</code> · <code>OrdersController.cs</code> · <code>OrderService.cs</code> · <code>AuthService.cs</code> · <code>AdminOrderService.cs</code>

Mẹo: nói chậm ở ba chỗ — dòng 34–35 của `OrderService` (server tự tính giá), chỗ hash BCrypt, và chỗ giải mã JWT. Ba chỗ đó là nơi thấy thấy mình hiểu hay không hiểu.

Nếu lỡ lời hay nói vấp thì **đừng quay lại từ đầu** — dừng 2 giây rồi nói lại câu đó, lúc dựng cắt đi là xong.